Modul 12

Tuning von Datenbanksystemen

Josef Altmann



MS Bing / Copilot generiert am 13. August 2024: „Generiere ein reales Bild mit einem springenden Wolf auf weißem Hintergrund.“



Der vorliegende Foliensatz basiert auf:

Winand, M.: SQL Performance Explained, ISBN: 978-3-9503078-1-8, www.winand.at (2012)
Winand, M.: Use the Index, Luke! Anleitung zur SQL-Performance für Entwickler
Kleppmann, M.: Datenintensive Anwendungen designen, O'Reilly, 1. Auflage (2019)
Shasha, D., Bonnet, P.: Database Tuning, Kaufmann (Elsevier), 2nd Edition (2003)

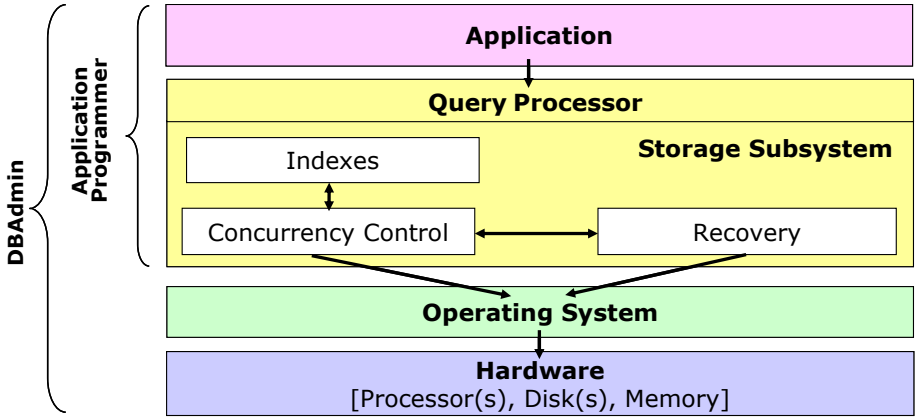
Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Begriffsbestimmung

- Datenbank-Tuning umfasst ...
  - ... sämtliche Aktivitäten, die der Verbesserung der Zeiteffizienz (response time) bzw. des Durchsatzes (throughput) einer Datenbank-Anwendung dienen.
- Tuning ...
  - Datenbankanwendung
    - Connection Pooling
    - Precompiled Queries
    - Lazy Loading
  - Datenbank
    - Schema (Datenbankdesign)
    - Anfragen
    - Interne Strukturen (Indizes)
    - Transaktionen
    - Recovery
  - Datenbankumgebung
    - Hauptspeicher, Sekundärspeicher, Netzkomponenten



➢ In 50 % der Fälle sind die Performance-Probleme bereits im Design der Applikation präsent!

Tuning Prinzipien

- Think globally, fix locally
  - Does it matter?
- Partitioning breaks bottlenecks
  - Temporal and spatial
- Start-up costs are high, running costs are low
  - Disk transfer, cursors, query parsing
- Render onto server what is due onto server
  - Server vs. client processing
- Be prepared for trade-offs
  - Indexes and inserts

© 2026

Tuning Prinzipien

Think globally, fix locally

- Gesamtheitliche und genaue Problemanalyse/-identifikation
  - Diagnose
- Geringstmögliche Änderung für Problembehebung
  - Therapie

- Beispiele:
  - langsam bearbeitete Anfrage
    → vor Tuning Wichtigkeit (bspw. Häufigkeit) überprüfen
  - ausgelastete Platte
    → vor Neukauf prüfen,
    - » ob Indizes verwendet werden bzw. für die Anfragen geeignet sind
    - » ob Log- und Nutzdaten auf einer Platte verwaltet werden

Tuning Prinzipien

Partitioning breaks bottlenecks

- Engpass: voll ausgelastetes Betriebsmittel begrenzt den Durchsatz
- Partitionierung: Aufteilung der Arbeit, um voll ausgelastetes Betriebsmittel zu verhindern
- Partitionierungsarten:
  - Räumlich: verschiedene, physisch getrennte Systeme; z.B. auf die Filialen oder Abteilungen eines Unternehmens, Tochterfirmen eines Konzerns
  - Zeitlich: Scheduling nach Transaktionsklassen; z.B. lange Batch-Läufe außerhalb der online Betriebsläufe
  - Logisch: nach disjunkten Sperrbedarfen

© 2026

Tuning Prinzipien

Startup costs are high, running costs are low

- Operationen haben Fixkosten (vgl. Rüstkosten bei der Produktion) und variable Kosten → möglichst gute Nutzung der Fixkosten; z.B:
  - für Nachrichten Paketgrößen ausnutzen: Header wird unabhängig von der Länge des Inhalts gebraucht; Latenzzeit für Versenden einer 1-Byte-Nachricht ist fast gleich groß wie für 1 Kbyte
  - Prekompilierung/Caching häufig ausgeführter Anfragen: Übersetzungszeit ist relativ hoch (> 10.000 Instruktionen; Parsen, Optimieren, Berechnen der Zugriffspfade)
  - große Transfereinheiten (Tupelmengen) bei Client/Server-Interaktion: die einzelne Interaktion ist teuer (rpc, Netzwerk)

Tuning Prinzipien

Render onto server what is due onto server

- Entwurfsentscheidung, abhängig von
  - Ressourcen auf Server- bzw. Client-Seite
  - gewünschte/erforderliche Lokalisierung relevanter Information
- Typische Entwurfsentscheidungen:
  - Joins am Server
  - Benutzerinteraktion am Client (außerhalb von Transaktionen)
  - für rechenintensive Anwendungen: Datenpuffer in Client-Workstation
  - Trigger statt Polling

© 2026

Tuning Prinzipien

Be prepared for trade-offs

- Steigerung des Durchsatzes/Antwortzeit kann Kosten für
  - Hauptspeicher
  - Festplattenkapazität
  - CPUs

  erhöhen!

„You want speed? How much are you willing to pay?“

- Beispiel:
  - Indizes verringern die Laufzeit von kritischen Abfragen – ABER
    - zusätzliche Festplattenkapazität ist notwendig
    - Indizes verlangsamen INSERT und UPDATE

Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Tuning – Transaktionen 1/6

- "ideale" Transaktion
  - fordert wenige Sperren an
  - bevorzugt Lese- gegenüber Schreibsperren
  - sperrt in kleiner Granularität
  - hält Sperren für geringstmögliche Zeit

  um Sperrkonflikte zu minimieren

- Ziel des Tunings
  - Performanz, d.h.
    - Verzögerungen reduzieren (T1 muss auf Freigabe von Sperren durch T2 warten)
    - Deadlocks vermeiden
  - Korrektheit, d.h.
    - garantieren, dass jede Transaktion (scheinbar) isoliert ausgeführt wird (Serialisierbarkeit, Isolationsebne)

„Trade-off between performance and correctness!“

Tuning – Transaktionen 2/6

- Verwenden von lesekonsistenten Sichten
  - Snapshot Isolation: lesende Zugriffe setzen keine Sperren
- Eliminieren unnötiger Sperren, z.B.
  - wenn nur eine Transaktion läuft wie etwa beim Laden einer Datenbank
  - wenn alle Transaktionen nur lesen, wie z.B. bei OLAP-Systemen
- Aufspalten von Transaktionen
  - Ziel: kleinere, serialisierbare Einheiten mit kürzeren Sperren
  - Voraussetzung: alle nebenläufig auszuführenden Transaktionen sind bekannt, d.h. keine adhoc-Transaktionen
- Reduktion der Isolationstufe

© 2026

Tuning – Transaktionen 3/6

- Wahl der geeigneten Sperrgranularität
  - Satz-Sperren, wenn auf einzelne bzw. wenige Sätze zugegriffen wird, die sich in der Regel auf verschiedenen Seiten befinden
  - Seiten- oder Tabellen-Sperren für lange Änderungs-transaktionen → weniger Deadlocks und geringerer Overhead
  - Lock Escalation, wenn Treshhold erreicht
- Schema-Änderungen (DDL-Locks) nur in Zeiten geringer Auslastung
- Umgehen von "Hot Spots" durch
  - Partitionierung
  - Zugriff so spät wie möglich durch Transaktion

Tuning – Transaktionen 4/6

- Aus Anwendersicht akzeptabel oder aus Performanzgründen erforderliche niedrigere Isolation
  - Aufspaltung von Transaktionen, die Sperren während Benutzerinteraktionen halten:

Beispiel: Flugreservierung

1. Lesen verfügbare Sitze
2. Verkaufsgespräch mit Kunde
3. Sitzreservierung

} Unakzeptable Performanz, wenn in einer Transaktion realisiert

Lösung: (1) und (3) jeweils eigene Transaktion
(2) außerhalb des Transaktionskontexts

Problem: in (1) ermittelter und in (2) vom Kunden gewünschter Sitz könnte bei (3) bereits vergeben sein → muss akzeptiert werden

Tuning – Transaktionen 5/6

- Aus Anwendersicht akzeptabel oder aus Performanzgründen erforderliche niedrigere Isolation
  - Tolerierung von Lücken in eigentlich dichten Wertebereichen (Zählern, Tickets etc.):

    Beispiel: Auftragsnummernvergabe (Nummern sollen nach Eingangsreihenfolge fortlaufend vergeben werden)
    → globaler Zähler ist Flaschenhals

    Lösung:
    - Systemeinrichtungen (z.B. Oracle: Sequence) geben Zähler unmittelbar nach Inkrementierung frei
    - bei Programmierung: Lesen und Erhöhung des Zählers in eigener Transaktion

    Problem:
    - bei Abbruch von Transaktionen werden u.U. manche Nummern nicht vergeben

  - Reduktion des Isolationsgrades, wenn bei statistischen Auswertungen ungefähre Ergebnisse ausreichend sind

Tuning – Transaktionen 6/6

Zusammenfassung

- Finde die Balance zwischen sicherer, konsistenter Verarbeitung einerseits und der Schwere von Fehlern beim Aufweichen der Isolationsebenen
- Richtige Einstellung der Sperrgranularität: z.B. Möglichkeit, eine komplette Relation zu sperren, falls viele Tupel zugegriffen werden, anstatt alle Tupel einzeln zu sperren
- Transaktionen sollen nicht zu lange dauern (relationale Systeme sind nur für kurze Transaktionen ausgelegt)
- Schema-Änderungen mittels DDL-Anweisungen oder Reorganisationsläufe sollen nicht während des operativen Betriebes durchgeführt werden.

© 2026

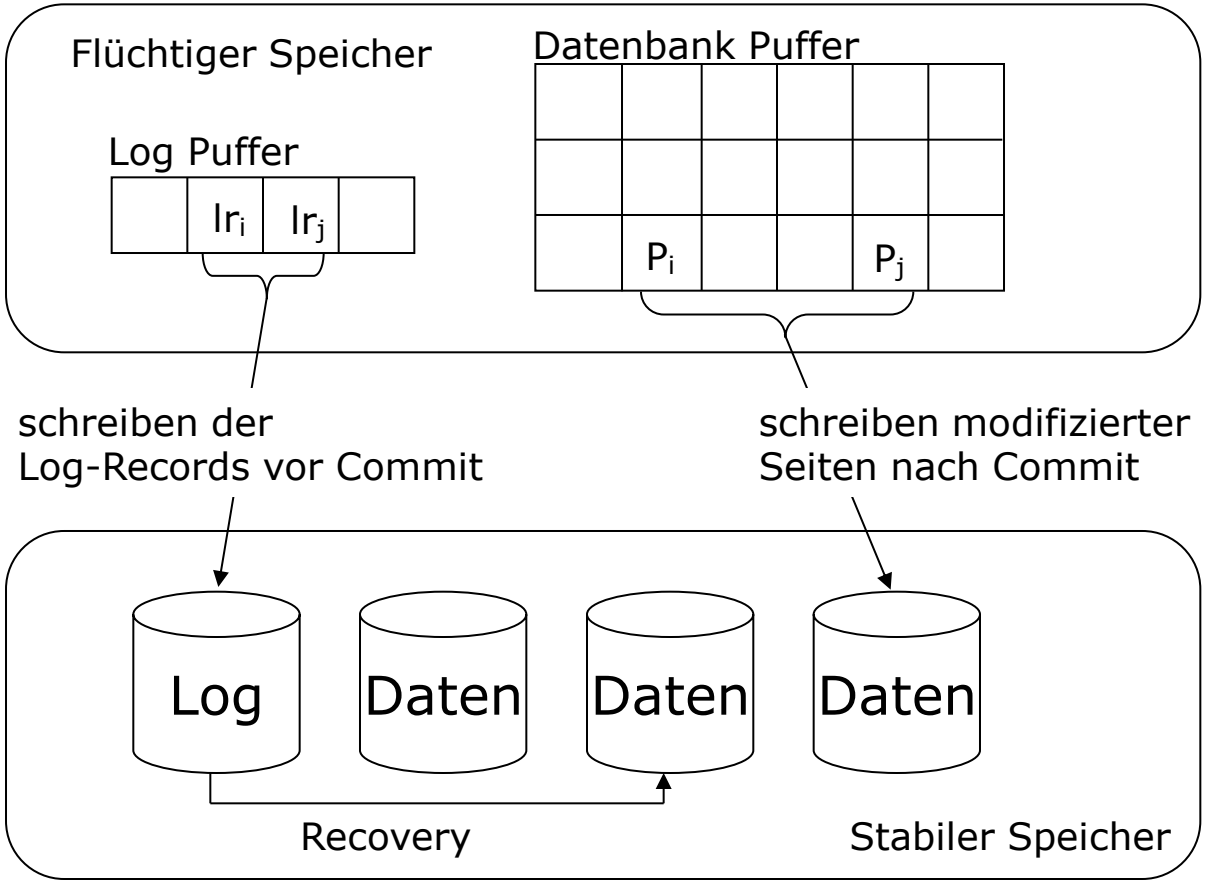
Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Tuning – Logging und Recovery 1/3

- Aus Performanzsicht reiner Overhead, aber notwendig für Dauerhaftigkeit und Atomarität von Transaktionen



Tuning – Logging und Recovery 2/3

- Maßnahmen Recovery-System/Log:
  - Log auf eigener Platte (Vermeidung von Kopfpositionierungen, zusätzliche Sicherheit z.B. bei Crash‘s)
  - Reduktion der Größe großer Update-Transaktionen (Aufspalten)
  - Zeitdauer für Recovery und Zeit-/Platzoverhead für Logging/Durchschreiben (Checkpoints, DB-Dumps) abwägen
- Commit-Strategie
  - buffered commit vorteilhaft, da Schreiben ins Log wesentlich schneller (Log ist sequentielle Datei) und DB-Schreiboperationen eher zufallsverteilt

© 2026

Tuning – Logging und Recovery 3/3

- bei hohem Änderungsvolumen group commit vorteilhaft: gemeinsames Schreiben auf Log für mehrere Transaktionen

- Aufspalten änderungsintensiver Batch-Transaktionen in Mini-Batch-Transaktionen (Batch to Mini-Batch), falls Nebenläufigkeit unproblematisch
  - Beispiel:

    Transaktion, die alle Konten mit Bewegungen am gegebenen Tag aktualisiert

    → aufspalten in `n` Transaktionen, die disjunkte Kontenmengen bearbeiten

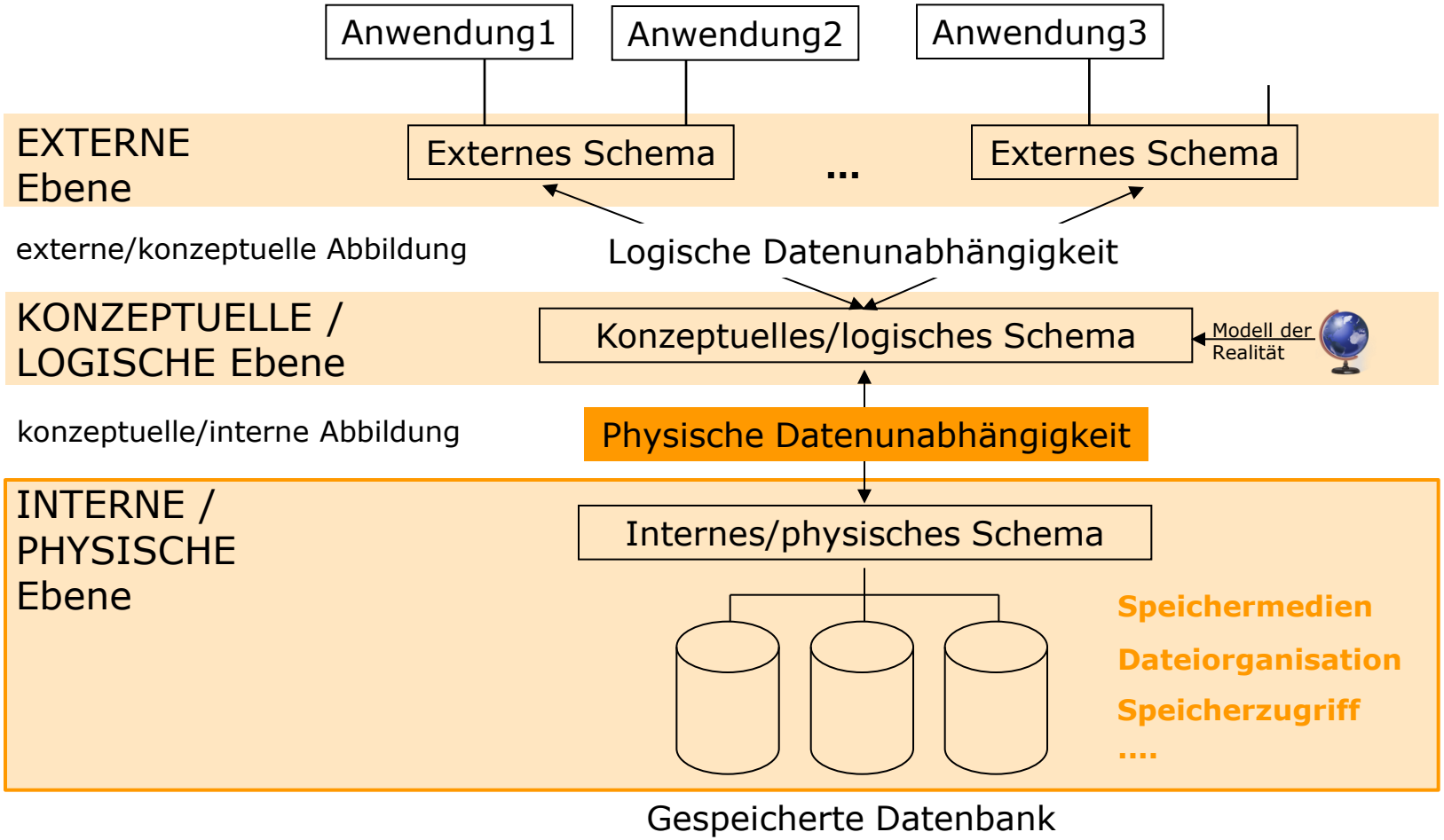
    → effektiveres Recovery, höhere Nebenläufigkeit, geringere Pufferbelastung

Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Tuning – Physische Datenorganisation 1/7



- Änderungen an der physischen Speicher- oder der Zugriffsstruktur (beispielsweise das Anlegen oder Entfernen einer Indexstruktur) hat keine Auswirkungen auf das logische/konzeptuelle Schema.

Tuning – Physische Datenorganisation 2/7

Zugriffs- und Indexstrukturen

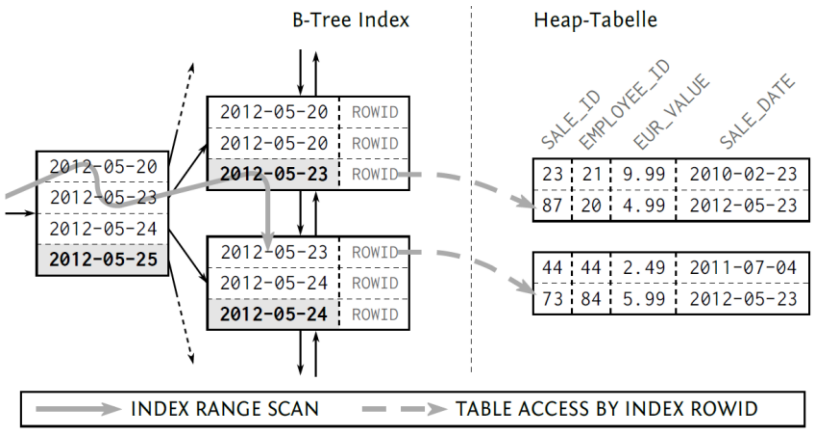
- Indizes beschleunigen Punkt-, Bereichs- und Verbundanfragen erheblich.
- Indizes verlangsamen das Einfügen, Löschen und Ändern von Tupeln der Relation.
  - Indizes müssen automatisch nachgeführt werden.
- Indizes benötigen Speicherplatz und aktuelle Statisiken
- Indexerzeugung kann lange dauern.
  - Bulk Load: Es ist (viel) effizienter, zuerst die Daten in die Tabelle einzufügen und nachher alle Indizes zu erstellen als umgekehrt.

- Indizesdefinition ist eine der schwierigsten Aufgaben des Datenbankdesigns
  - Vorhersage der Query Workload und Änderungsfrequenz
  - Attributwahl
    - häufiger Vergleich mit Konstanten
    - häufiges Verbundprädikat

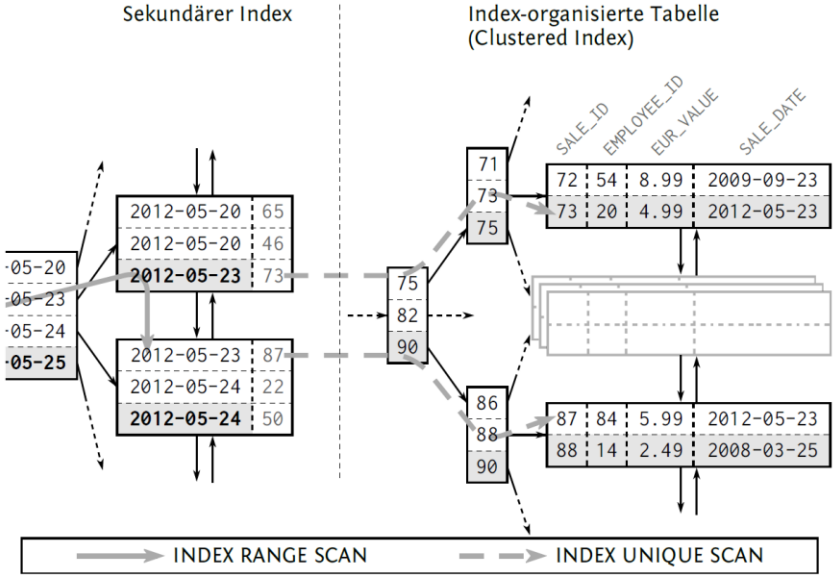
Tuning – Physische Datenorganisation 3/7

Dateiorganisation

- Speicherungsform der Datensätze in der Datendatei bestimmt die Effizienz der Zugriffs- und Modifkations-operationen auf den Datensätzen.
  - unsortierte Speicherung: Heap-organisierte Tabelle
  - sortierte Speicherung: Index-organisierte Tabelle (IOT)
  - gestreute Speicherung: Hash-organisierte Tabelle



Indexgestützter Zugriff auf eine Heap-Tabelle
[Winand, M.: SQL Performance Explained (2012), S. 123]



Sekundärer Indexzugriff auf eine IOT
[Winand, M.: SQL Performance Explained (2012), S. 124]

➢ üblich: Heap-organisierte Tabellen

Tuning – Physische Datenorganisation 4/7

Dateiorganisation – Temporäre Tabelle

- **Temporäre Tabelle** ...
  - dient als Arbeits- oder Puffertabelle für
    - komplexe Anfragen, um Ergebnisse von Unteranfragen zu speichern und wiederzuverwenden
    - Data Warehouses, um Zwischenergebnisse aus Teiltabellen zu speichern und weiterzuverarbeiten
    - die Vermeidung von Synchronisationsoperationen (Sperren) zwischen Sessions
    - die automatische Speicherfreigabe durch das Datenbanksystem am Transaktions- bzw. Sessionende.
  - wird als Datenbankobjekt im Data Dictionary abgelegt
  - kann entweder lokal (nur für die aktuelle Session) oder global (für alle Sessions) sichtbar sein
  - speichert keine aufwendigen Fehlererholungsinformationen (kein Redo, nur Undo)

- **Datensätze** einer Temporären Tabelle ...
  - werden nicht dauerhaft im Datenbanksystem gespeichert
  - existieren bis zum `COMMIT` einer Transaktion oder spätestens bis zum Sessionende

© 2026

Tuning – Physische Datenorganisation 5/7

Oracle: Temporäre Tabelle

- Oracle unterstützt
  - GLOBAL TEMPORARY TABLE (ab Oracle 8i)
  - PRIVATE TEMPORARY TABLE (ab Oracle 18c)

CREATE GLOBAL TEMPORARY TABLE-Anweisung

```
CREATE GLOBAL TEMPORARY TABLE Table_Name (
   Column_Definition_n,
   ...,
   Table_Constraints_n
) ON COMMIT [DELETE ROWS | PRESERVE ROWS];
```

- Jede Session sieht nur jeweils ihre eigenen Datensätze.
- Datensätze einer temporären Tabelle werden bei
  - `ON COMMIT DELETE ROWS` (Default) nach `COMMIT` in der Transaktion
  - `ON COMMIT PRESERVE ROWS` spätestens bei Sessionende

  automatisch gelöscht.

Tuning – Physische Datenorganisation 6/7

Oracle: Temporäre Tabelle - Beispiel

CREATE TABLE-Anweisung (Basistabelle)

```
CREATE TABLE Produkt (
    ProId INTEGER GENERATED ALWAYS AS IDENTITY(START WITH 1 INCREMENT BY 1
                MINVALUE 1 MAXVALUE 1000000 NOCYCLE),
    Code VARCHAR2(100) NOT NULL UNIQUE,
    Bezeichnung VARCHAR2(250) NOT NULL,
    Preis DECIMAL (9,2));
```

CREATE GLOBAL TEMPORARY TABLE-Anweisung (Transaktionsbasiert)

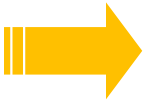
```
CREATE GLOBAL TEMPORARY TABLE ProduktBerechnung (
    ProId INTEGER,
    Code VARCHAR2(100) NOT NULL UNIQUE,
    Preis DECIMAL (9,2))
ON COMMIT DELETE ROWS;
```

INSERT-Anweisung

```
INSERT INTO ProduktBerechnung (ProId, Code, Preis)
SELECT ProId, Code, Preis
FROM Produkt WHERE Code LIKE 'AB%L';
```

COMMIT-Anweisung

```
COMMIT;
SELECT * FROM ProduktBerechnung;
```



Ergebnis

PROID	CODE	PREIS

Tuning – Physische Datenorganisation 7/7

Temporäre Tabelle versus Sicht

- Sicht
  - Datenabstraktion unabhängig von der zugrunde liegenden Tabellenstruktur
  - Sicherheit und Datenschutz durch eingeschränkten Datenzugriff
  - Wiederverwendbarkeit von komplexen Anfragen
  - Lesbarkeit durch Zerlegung von komplexen Anfragen in kleinere Teilanfragen

- Temporäre Tabelle
  - Zwischenspeicherung von Daten, die nicht über längere Zeiträume verwaltet/gespeichert werden müssen (bspw. vorübergehende Daten, ETL-Daten)
  - Performance-Optimierung durch Einsatz von Indizes und Statistiken
  - Verarbeitung von Daten innerhalb einer Transaktion/Session
  - Automatische Speicherfreigabe durch Datenbanksystem

© 2026

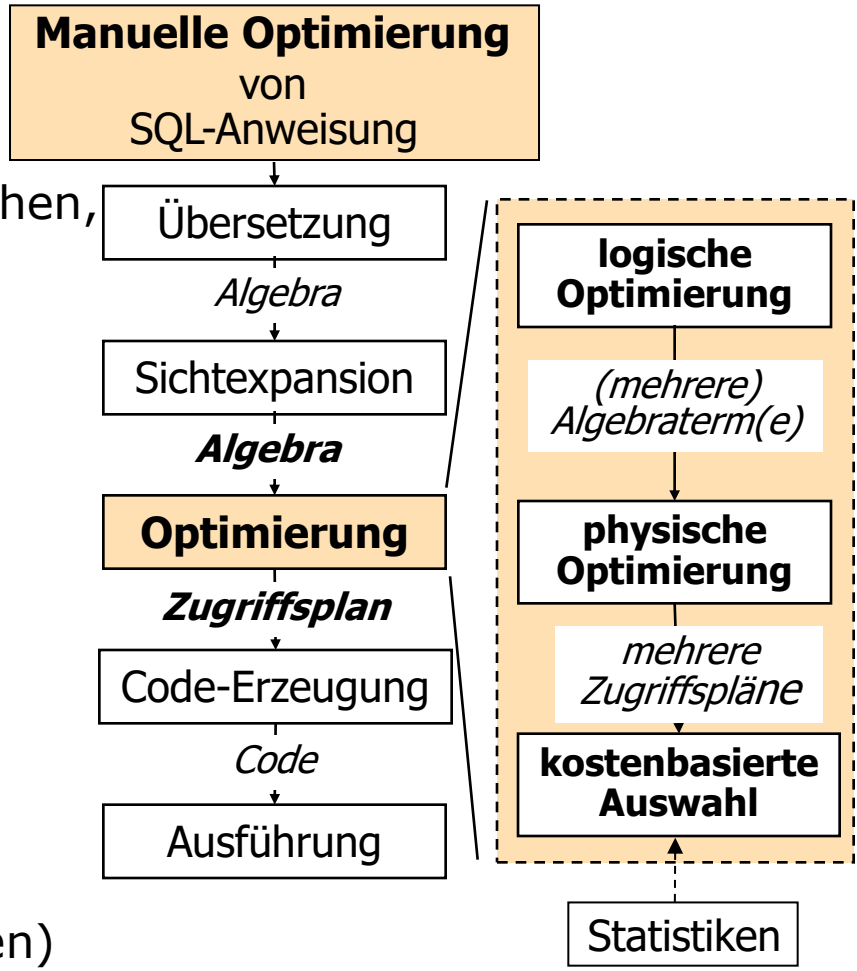
Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Tuning – Anfragen 1/9

Phasen der Anfragebearbeitung

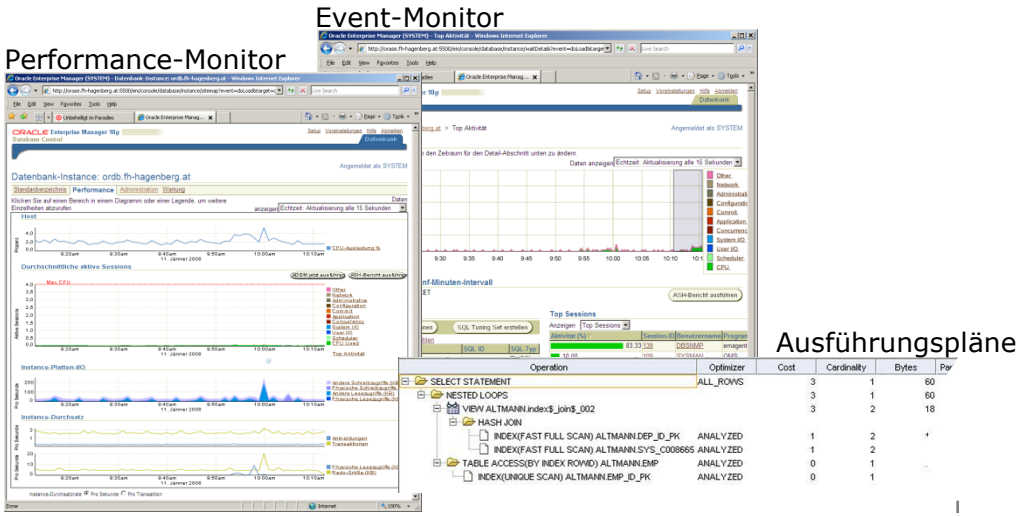


- Übersetzung und Sichtexpansion
  - arithmetische Ausdrücke vereinfachen, Unteranfragen auflösen, Einsetzen der Sichtdefinition
- Logische (algebraische bzw. Regel-basierte) Optimierung
  - Anfrageplan unabhängig von Speicherform umformen
- Physische (interne) Optimierung
  - konkrete Speicherformen (Indexe, Cluster) berücksichtigen
  - mehrere alternative interne Pläne
- Kostenbasierte Auswahl
  - Statistikinformationen (Größe von Tabellen, Selektivität von Attributen) für die Auswahl eines konkreten internen Plans nutzen
- Code-Erzeugung
  - wandelt Zugriffsplan in ausführbaren Code um

Tuning – Anfragen 2/9

Manuelle Optimierung von SQL-Anfragen

- Ziel
  - Umschreiben von SQL-Anweisungen zur Erhöhung der Performanz. Zwei unterschiedliche Ziele:
    - Antwortzeit: "Liefere erstes Tupel so schnell wie möglich"
    - Durchsatz (Standard): "Liefere alle Tupel so schnell wie möglich"
- Vorteil
  - Auswirkungen betreffen nur die jeweilige Anfrage, d.h. keine Seiteneffekte auf andere Anwendungen
- Ausgangspunkt
  - Ausführungspläne, Performance-Monitore, Event Monitore, …



Tuning – Anfragen 3/9
Ursachen schlechter Anfrage-Performanz

- Index-Verwendung wird verhindert durch
  - Berechnung auf indiziertes Attribut in der **WHERE**-Klausel
    … **WHERE Salary/12 > 45000** …
  - Funktion auf indiziertes Attribut in **WHERE**-Klausel
    … **WHERE SUBSTR(First_Name,1,1) = 'M'** …
    … **WHERE UPPER(Last_Name) = 'ALTMANN'** …
  - Ausdruck mit unterschiedlichen Datentypen → implizite Typ-Konvertierung **char_column = number_expression**
    wird von Oracle automatisch konvertiert in:
    TO_NUMBER(char_column) = number_expression
  - Verwendung von **!=** oder **IS NULL** oder **IS NOT NULL** oder **NOT** bei einem indizierten Attribut
  - verschachtelte Anfragen, in denen **IN** vorkommt, anstatt Verbundoperation

```
EMPLOYEE (EmpId, First_Name, Last_Name, Salary, ManagerId, DeptId)
DEPARTMENT (DeptId, Dept_Name, ManagerId, Location)
```

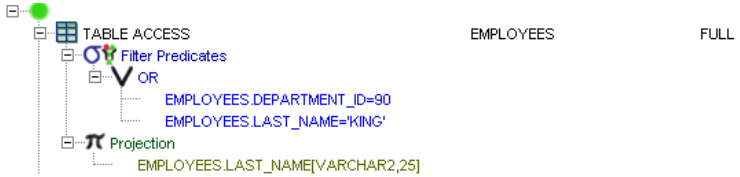
```
SELECT EmpId
FROM Employee
WHERE DeptId IN (SELECT DeptId FROM Department);
```

Tuning – Anfragen 4/9

Ursachen schlechter Anfrage-Performanz

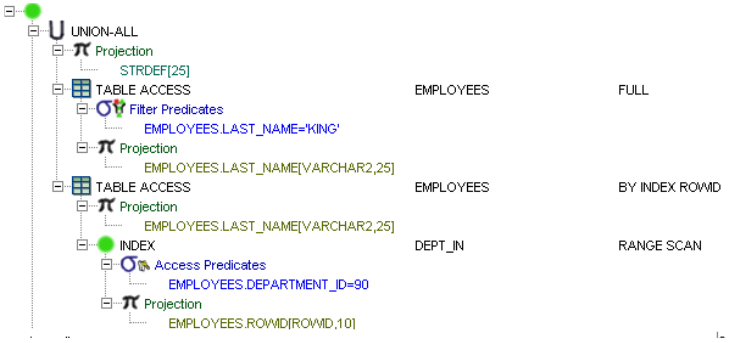
- Verwendung von **OR** anstatt von **UNION**, sofern beide Spalten indiziert sind

```
SELECT Emp.EmpId
FROM EMPLOYEE Emp
WHERE Emp.Last_Name = 'Altmann'
OR Emp.DeptId = 90;
```



Ausführungsplan - ORACLE

```
SELECT Emp.EmpId
FROM EMPLOYEE Emp
WHERE Emp.Last_Name = 'Altmann'
UNION ALL
SELECT Emp.EmpId
FROM EMPLOYEE Emp
WHERE Emp.DeptId = 90;
```



Ausführungsplan - ORACLE

UNION erfordert Sortierung, daher – wenn nicht notwendig - **UNION ALL** vewenden

Tuning – Anfragen 5/9

Ursachen schlechter Anfrage-Performanz

- Unnötiges **DISTINCT**

```
SELECT DISTINCT EmpId
FROM EMPLOYEE
WHERE DeptId = 90;
```

DISTINCT ist unnötig, da **EmpId**-Schlüssel in **EMPLOYEE** und daher auch Schlüssel einer Teilmenge dieser Relation ist.

```
SELECT DISTINCT EmpId
FROM EMPLOYEE INNER JOIN
     DEPARTMENT USING(DeptId);
```

DISTINCT ist unnötig, da jedes **EMPLOYEE**-Tupel mit höchstens einem **DEPARTMENT**-Tupel verbunden wird (**DeptId** ist dort Schlüssel!)

© 2026

Tuning – Anfragen 6/9
Ursachen schlechter Anfrage-Performanz

- Korrelierte Anfragen

```
SELECT Emp_1.EmpId, Emp_1.DeptId
FROM EMPLOYEE Emp_1
WHERE Salary =
    ( SELECT MAX(Salary)
      FROM EMPLOYEE Emp_2
      WHERE Emp_2.DeptId = Emp_1.DeptId );
```

Bestbezahlte/r Mitarbeiter*in pro Abteilung

- besser: Verwendung einer temporären Relation

```
CREATE GLOBAL TEMPORARY TABLE
BestEarner
(  DeptId NUMBER(2) UNIQUE,
   MaxSalary NUMBER (9,2))
ON COMMIT PRESERVE ROWS;
```

```
INSERT INTO BestEarner (DeptId, MaxSalary)
SELECT DeptId, MAX(salary)
FROM Employees
WHERE DeptId IS NOT NULL
GROUP BY (DeptId);
```

```
SELECT Emp.EmpId, Emp.DeptId
FROM EMPLOYEE Emp INNER JOIN BestEarner Best
ON(Emp.DeptId = Best.DeptId AND Salary = MaxSalary);
```

© 2026

Tuning – Anfragen 7/9
Ursachen schlechter Anfrage-Performanz

- Unnötige Verwendung der **HAVING**-Klausel

```
SELECT AVG(Salary), DeptId
FROM EMPLOYEE
GROUP BY DeptId
HAVING DeptId = 90;
```

- besser:

```
SELECT AVG(Salary) AS Dept_90_AVG_Salary
FROM EMPLOYEE
WHERE DeptId = 90;
```

```
SELECT AVG(Salary), DeptId
FROM EMPLOYEE
WHERE DeptId = 90
GROUP BY DeptId;
```

© 2026

Tuning – Anfragen 8/9

Ursachen schlechter Anfrage-Performanz

- Verbund - Unterabfragen
  - Verbund – `EXISTS` - `IN`
    - Meistens Verbund schneller als Unterabfragen
    - Meistens korrelierte Unterabfrage schneller als nicht-korrelierte (`EXISTS` schneller als `IN`)
  - `NOT EXISTS` - `NOT IN`
    - Meistens korrelierte Unterabfrage schneller als nicht-korrelierte (`NOT EXISTS` schneller als `NOT IN`)
- Sortierungen
  - Sortierungen - wenn möglich - vermeiden
    - Sortierung: `DISTINCT, GROUP BY, ORDER BY, UNION`,...
    - `UNION ALL` statt `UNION`

Tuning – Anfragen 9/9

Beispiel: EXISTS versus IN

Ausgangsstatement	Optimiertes Statement
SELECT e.employee_id, e.first_name, e.last_name, e.salary FROM EMP e WHERE EXISTS (SELECT 1 FROM ORDERS o WHERE e.employee_id = o.sales_rep_id AND o.customer_id = 144);	SELECT e.employee_id, e.first_name, e.last_name, e.salary FROM EMP e WHERE e.employee_id IN (SELECT o.sales_rep_id FROM ORDERS o WHERE o.customer_id = 144);
SELECT e.employee_id, e.first_name, e.last_name, e.department_id, e.salary FROM EMP e WHERE e.department_id = 80 AND e.job_id = 'SA_REP' AND e.employee_id IN (SELECT o.sales_rep_id FROM ORDERS o);	SELECT e.employee_id, e.first_name, e.last_name, e.salary FROM EMP e WHERE e.department_id = 80 AND e.job_id = 'SA_REP' AND EXISTS (SELECT 1 FROM ORDERS o WHERE e.employee_id = o.sales_rep_id);

Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Tuning des Schemas

Datenbankentwurf

- Datenbankdesign sollte normalerweise nicht für Tuning-Zwecke geändert werden
- Mögliche Tuningmaßnahmen
  - Denormalisierung des Datenbankschemas
    - Relationen werden wieder zusammengefasst
      → mögliche Update-Anomalien
  - Einführung eines künstlichen Integer-Primärschlüssels
  - Anpassung von Datentypen
    - Verbesserung z.B. durch Zeichenketten fester Länge

© 2026

Überblick

- Begriffsbestimmung und Prinzipien
- Tuning – Transaktionen
- Tuning – Logging und Recovery
- Tuning – Physische Datenorganisation
- Tuning – Anfragen
- Tuning – Datenbankschemas
- Zusammenfassung

© 2026

Zusammenfassung
Methodisches Tunen

- **Phase 1:** Optimierung von SQL-Anweisungen und -Zugriffen
  - Manuelle Optimierung
  - Zugriffsstrukturen (Indizes, Cluster)
- **Phase 2:** Richtige Konfigurierung der Speicherbereiche
  - Shared Pool- und Datenpuffer-Größen, etc.
- **Phase 3:** Optimieren der verbleibenden physischen Zugriffe
  - Gleichmäßige I/O-Verteilung auf verschiedene Platten
  - Konfiguration der Extents sowie der DB-Blöcke
- **Phase 4:** Vermeidung von Zugriffskonflikten
  - Größe u. Anzahl der Rollback-Segmente
  - Zugriffskonflikte auf Datenblöcke u. Redo Log Buffer

© 2026